智元D1开发教程｜Sim2Real全指南（上）——从控制接入到真机落地

原创 敢敢不敢写bug 敢敢AUTOHUB 2026年4月16日 08:45 上海 听全文

智元D1开发教程｜Sim2Real全指南（上）——从控制接入到真机落地

本系列致力于教会大家如何使用智元D1跑通整个Sim2Real流程，如果想要进一步了解智元D1二开内容，请扫描下方二维码或点击链接获取：



> *链接：*
> *https://agirobot.feishu.cn/share/base/form/shrcnvprcMGYXXBy0vS9kru3bSf*

1. 引言

在强化学习的开发流程里，训练完成常常会带来一种错觉：最难的部分似乎已经过去了。模型收敛了，奖励曲线涨起来了，机器人在仿真中也已经能够稳定站立、行走，甚至完成转向，看上去距离落地只差“最后一步”。但真正把策略接到实物机器人上时，很快就会发现，训练结束并不意味着任务结束，恰恰意味着另一段更具挑战性的工作才刚刚开始。



极速写方案！千问电脑端让办公提速翻倍。
千问-阿里AI助手
立即体验

原因并不复杂。训练完成，只能说明策略已经在**训练环境设定的规则**里学会了动作逻辑；而真机部署要验证的，则是这套逻辑在真实世界中是否依然成立。把一个 `.pt` 文件拷进机器人、启动程序、观察现象，这只是表面步骤。更深一层看，实机部署真正考验的是整条控制链路：观测是否可信、命令是否连续、执行是否稳定、系统是否能承受现实中的噪声、延迟与建模误差。

仿真之所以适合训练，是因为它足够“可控”。机器人的质量分布、关节阻尼、摩擦系数、地面接触、传感器反馈以及控制频率，几乎都可以被精确建模、固定下来，并且反复复现。在这样的环境里，策略面对的是一个相对理想、边界清晰的系统。可一旦进入真机，这种理想状态就不复存在了。IMU 会有噪声，关节反馈会有误差，姿态估计会受振动与冲击干扰，线速度有时甚至不是一个可以稳定直接获得的量。与此同时，动作执行也不再像仿真中那样“指哪打哪”。策略在训练中学到的，是一套建立在“系统基本按预期响应”前提下的控制逻辑；而真机部署要解决的问题，是如何让这套逻辑在偏差、延迟、噪声和不确定性同时存在的条件下，仍然能够工作。

所以，Sim2Real 从来不是“训练结束后的附加步骤”，而是强化学习走向真实机器人必须跨过去的一道门槛。对于智元 D1 来说，这道门槛并不只体现在算法层，更体现在 SDK 接入方式、控制循环频率、状态重组方法、部署链路检查以及真机调参策略上。上篇就从这些最关键、也最容易踩坑的部分开始，系统梳理一遍 D1 的 Sim2Real 落地过程。

2. 智元 D1 在 Sim2Real 中的控制方式

2.1 SDK 架构分析

先从整体系统架构看起。图 1 中最核心的一层，并不是最底部的硬件，也不是最上层的云端，而是中间的 **Robot Interface**。它可以理解为整台机器的软件中枢：向上连接 APP、遥控手柄、SDK 以及各类应用能力，向下连接系统接口、硬件控制与数据预处理，再进一步对接相机、IMU、深度相机、算力平台和执行机构。

这意味着，无论外部是通过手柄控制、APP 控制，还是通过二次开发 SDK 介入，最终都不是绕过系统、直接操纵硬件，而是先进入 Robot Interface 这一层，再由它把命令分发给下层运动控制或命令接口，同时把状态数据向上抽象成 SDK 和应用层能够消费的格式。换句话说，我们接入的并不是一台“裸机器人”，而是一套已经完成软件分层、具备系统约束的机器人平台。理解这一点，对后面选择 Highlevel 还是 Lowlevel 接口非常重要。

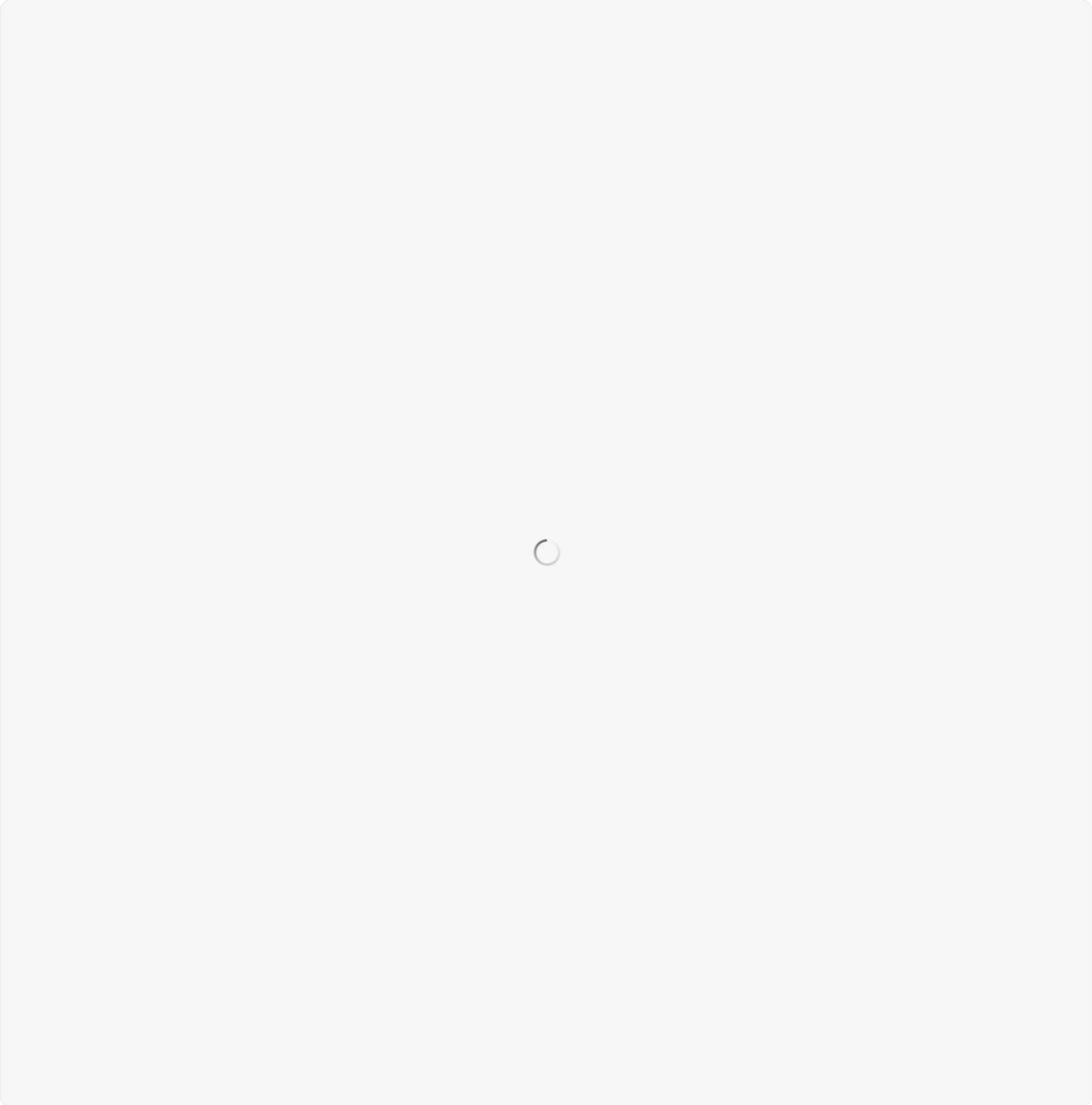


图1 整体系统架构图

再看图 2，Lowlevel 与 Highlevel 的分流关系就更加清晰了。SDK 与 Robot Interface 之间，主要存在四类数据流：`high_level_cmd`、`low_level_cmd / motor_cmd`、`motor_data` 和 `imu_data`。其中，高层控制命令会流向 MotionControl，也就是机器人原有的运动控制逻辑；底层控制命令则会流向

Command Interface，这才是 Lowlevel 真正进入执行链的入口。反过来，关节状态和 IMU 状态又会由 Robot Interface 回流给 SDK，形成完整的上行反馈链路。

因此，从控制语义上看，Highlevel 更像是“调用机器人已有的步态与能力”，而 Lowlevel 则是在“接管关节级控制入口，并同时拿到底层反馈状态”。对于强化学习部署来说，策略需要在每个控制周期内根据当前状态生成各关节的目标位置、目标速度、增益和前馈项，因此真正适合 Sim2Real 落地的，是 Lowlevel 这一条路径。

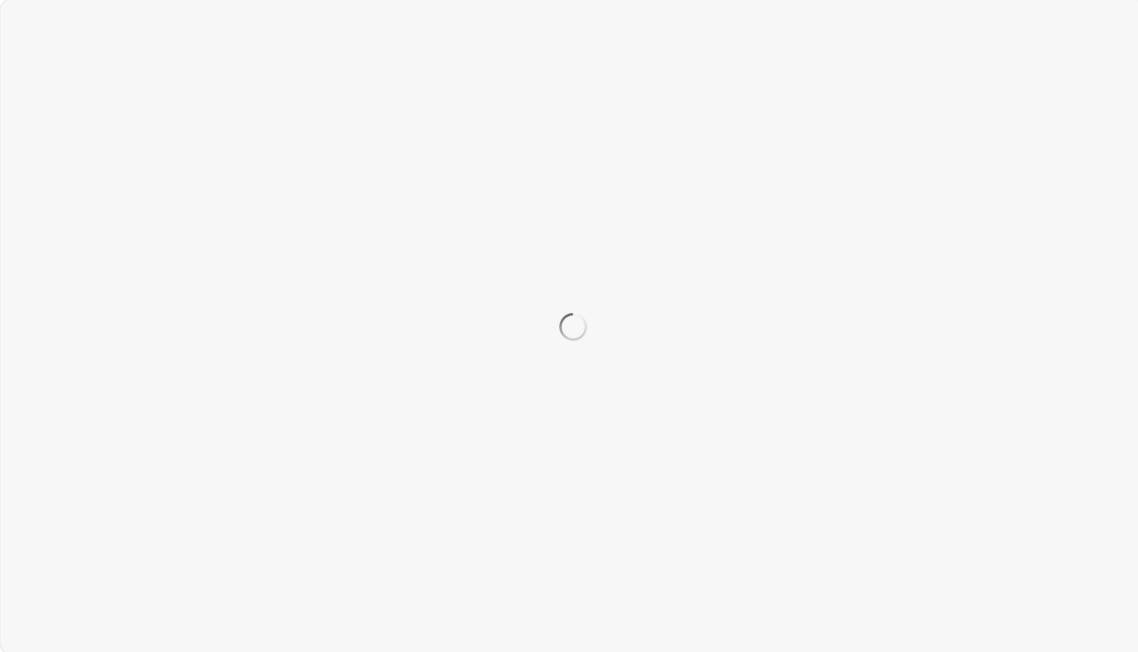


图2 SDK与interface关系

2.2 Lowlevel 控制接口的具体实现

依据官方在 `src/rl_sar/library/thirdparty/robot_sdk/agibot/agibot_D1_Edu-Ultra/include/zsl-1/lowlevel.h` 中提供的头文件，我们构造了 `rl_real_d1.cpp` 来调用 Lowlevel 接口，实现对智元 D1 的运动控制。下面分几个关键环节展开说明。

1）进程级别的 LowLevel-only 设计

在 `rl_real_d1.cpp` 的构造函数中，首先要做的一件事，就是明确将控制模式锁定为 **Lowlevel-only**。虽然 Highlevel 中也提供了一些工程接口，例如查询电池电量等，但官方已经明确说明，高低层模式之间的切换需要时间，因此在部署强化学习策略时，不建议混用两套控制链路。

```
// SDK manual states HighLevel and LowLevel must not be used at the same time.
// This RL runner is a pure LowLevel controller, so only initialize LowLevel here.
this->d1_lowlevel.initRobot(local_ip, local_port, robot_ip);
this->lowlevel_mode = true;
```

这样做的核心价值，在于保证控制权的唯一性。因为一旦 Highlevel 和 Lowlevel 同时存在于一个进程中，就有可能出现控制权冲突：一边是机器人原生运控在输出关节目标，另一边是策略网络也在生成关节目标，最后究竟哪一路命令真正送到底层电机，就会变得不可控。对真机来说，这种不确定性是非常危险的。

当然，LowLevel-only 的设计也意味着牺牲了部分灵活性，但在实机部署阶段，相比“功能尽可能多”，更重要的永远是“控制链路尽可能确定”。

与此同时，这里还把网络参数做成了启动参数，以便根据不同调试场景快速切换部署环境：

```
int local_port = 43988;
std::string local_ip = "192.168.234.2";
std::string robot_ip = "192.168.234.1";

if (argc >= 2)
{
    local_ip = argv[1];
}
if (argc >= 3)
{
    robot_ip = argv[2];
}
```

这意味着，无论是通过无线网络还是有线网络进行调试，都可以直接在启动阶段切换本机 IP 与机器人 IP，而不用反复修改源码，工程上会方便很多。

2）500Hz 持续流式下发：策略环与控制环分离

D1 这份实现里，一个非常值得强调的工程细节，是它把控制环与策略环拆成了两个不同频率的循环。

```
this->loop_keyboard = std::make_shared<LoopFunc>(
    "loop_keyboard", 0.05, std::bind(&RL_Real::KeyboardInterface, this));

this->loop_control = std::make_shared<LoopFunc>(
    "loop_control", 0.002, std::bind(&RL_Real::RobotControl, this));

this->loop_rl = std::make_shared<LoopFunc>(
    "loop_rl",
    this->params.Get<float>("dt") * this->params.Get<int>("decimation"),
    std::bind(&RL_Real::RunModel, this));
```

这三个循环各司其职：

- `loop_keyboard`：处理键盘或手柄输入，频率相对较低；
- `loop_rl`：执行策略推理，按照训练中的 `dt × decimation` 节奏运行；
- `loop_control`：真正负责与机器人交互，以 **0.002 秒，也就是 500Hz** 的频率持续下发控制命令。

为什么要这么设计？因为 D1 的 Lowlevel 底层不是“收到一次命令后就可以稳定保持很久”的模式，而是要求上层持续、高频地发送控制流。也就是说，不能简单理解成“推理一次，发一次命令，然后等下次推理”。如果真的这样做，而策略推理频率又低于底层控制链期望频率，那么机器人在两个控制周期之间就会出现命令断档，最终表现出来的，就是动作发飘、关节抖动，甚至整体控制不稳定。

因此，这里采用的是一种非常典型的“双频结构”：

- **策略环**负责以训练时的节奏产生新动作；
- **控制环**负责以 500Hz 的高频，把“当前最新命令”持续不断地送到底层。

从工程抽象上讲，这就是一个生产者—消费者模型：`RunModel()` 负责生产新的关节目标，`RobotControl()` 负责高频消费这些目标，并稳定发送给机器人。这样的结构，既保留了策略推理的节奏一致性，又满足了底层控制对高频连续命令流的要求。

3）重组 SDK 原始状态，统一到内部状态抽象

Lowlevel 接入的第一步，并不是发命令，而是先把机器人返回的原始状态读上来。

```
this->d1_motor_state = this->d1_lowlevel.getMotorState();

std::vector<float> quat = this->d1_lowlevel.getQuaternion();
std::vector<float> gyro = this->d1_lowlevel.getBodyGyro();
```

这里获取到的三类信息，恰好也是强化学习部署最核心的底层观测：

- 电机状态 `motorState`
- 姿态四元数 `quaternion`
- 机体角速度 `body gyro`

拿到这些原始数据之后，并不是直接把它们塞进策略网络，而是先统一写入 `RobotState<float>` 这一套 rl_sar 内部通用的状态抽象中。这样做的好处非常明显：上层推理逻辑只需要面向 `RobotState` 这一种统一语义，而不需要感知底层不同 SDK 的具体数据格式。

```
```

```
if (quat.size() >= 4)
{
    state->imu.quaternion[0] = quat[0]; // w
    state->imu.quaternion[1] = quat[1]; // x
    state->imu.quaternion[2] = quat[2]; // y
    state->imu.quaternion[3] = quat[3]; // z
}

if (gyro.size() >= 3)
{
    state->imu.gyroscope[0] = gyro[0];
    state->imu.gyroscope[1] = gyro[1];
    state->imu.gyroscope[2] = gyro[2];
}
```

到这里，D1 SDK 返回的原始电机与 IMU 数据，就被重新组织成了策略训练阶段所使用的状态语义，可以直接进入观测构造与模型推理过程。这一步看似只是"数据搬运"，但实际上是 Sim2Real 中非常关键的一层：它决定了部署期的观测含义，是否真正和训练期保持一致。

4）将策略输出装配成 SDK 可执行命令

上行链路负责把机器人当前状态读回来，下行链路则要把网络输出真正装配成 D1 可以执行的命令。这部分逻辑放在 `SetCommand()` 中完成。它接收 rl_sar 内部统一格式的 `RobotCommand<float>`，然后逐条腿、逐关节写入 D1 SDK 的 `motorCmd`：

```
this->d1_motor_cmd.q_des_abad[sdk_leg] = q_abad_clamped;
this->d1_motor_cmd.q_des_hip[sdk_leg]  = q_hip_clamped;
this->d1_motor_cmd.q_des_knee[sdk_leg] = q_knee_clamped;

this->d1_motor_cmd.qd_des_abad[sdk_leg] = command->motor_command.dq[base_idx + 0];
this->d1_motor_cmd.qd_des_hip[sdk_leg]  = command->motor_command.dq[base_idx + 1];
this->d1_motor_cmd.qd_des_knee[sdk_leg] = command->motor_command.dq[base_idx + 2];

this->d1_motor_cmd.kp_abad[sdk_leg] = command->motor_command.kp[base_idx + 0];
this->d1_motor_cmd.kp_hip[sdk_leg]  = command->motor_command.kp[base_idx + 1];
this->d1_motor_cmd.kp_knee[sdk_leg] = command->motor_command.kp[base_idx + 2];

this->d1_motor_cmd.kd_abad[sdk_leg] = command->motor_command.kd[base_idx + 0];
this->d1_motor_cmd.kd_hip[sdk_leg]  = command->motor_command.kd[base_idx + 1];
this->d1_motor_cmd.kd_knee[sdk_leg] = command->motor_command.kd[base_idx + 2];

this->d1_motor_cmd.tau_abad_ff[sdk_leg] = command->motor_command.tau[base_idx + 0];
this->d1_motor_cmd.tau_hip_ff[sdk_leg]  = command->motor_command.tau[base_idx + 1];
this->d1_motor_cmd.tau_knee_ff[sdk_leg] = command->motor_command.tau[base_idx + 2];
```

值得注意的是，在真正写入 SDK 之前，代码先对目标关节位置做了一次限位裁剪：

```
const float q_abad_clamped = std::clamp(q_abad, D1_ABAD_MIN, D1_ABAD_MAX);
const float q_hip_clamped  = std::clamp(q_hip,  D1_HIP_MIN,  D1_HIP_MAX);
const float q_knee_clamped = std::clamp(q_knee, D1_KNEE_MIN, D1_KNEE_MAX);
```

这一步的意义非常大。因为仿真中的策略输出虽然已经受过训练约束，但到了真机侧，仍然可能因为初始姿态偏差、观测漂移、模型不匹配等原因，给出偏激的目标值。如果不在最终下发前再做一次限位保护，那么一次异常输出就有可能直接冲击真实关节，带来安全风险。

同时，这一层也提供了良好的调试价值：一旦发生超限，程序会打印警告，告诉开发者是哪条腿、哪个关节、原始期望值是多少、最终被裁剪到了什么范围。这样一来，就能够快速定位问题究竟是来自策略输出过猛，还是来自配置文件中的参数设置不合理，从而进一步回头微调 `config.yaml`。

装配完成之后，命令才会真正发送出去：

```
int ret = this->d1_lowlevel.sendMotorCmd(this->d1_motor_cmd);
if (ret < 0)
{
    std::cout << LOGGER::WARNING << "Failed to send motor command" << std::endl;
}
```

因此，从代码结构上看，Lowlevel 的命令链路并不是简单的“网络输出 -> 直接写电机”，而是一条更完整、更安全的链路：

网络输出 -> 统一命令结构 -> D1 专属映射 -> 关节限位保护 -> SDK 下发

这条链路的存在，本质上就是在尽可能提高实机部署过程中的稳定性和可解释性。

3. 实机部署 Sim2Real

由于智元 D1 机身内部算力受限，官方并不建议直接在机身计算单元中部署强化学习算法。因此，一个更现实、也更稳妥的方案，是使用外部电脑连接 D1，并通过 UDP 持续发送 Lowlevel 控制信号来完成策略部署。

这种方式虽然看起来“没有做到完全集成”，但对于 Sim2Real 的第一阶段来说反而更合理。因为它把策略推理与机器人本体解耦开来，既方便调试，也便于在部署初期快

速定位问题：到底是模型的问题、观测的问题，还是通信和控制链路的问题。

3.1 部署前最后的检查

在真正把策略接到机器人身上之前，有几类问题必须提前排查。因为它们一旦存在，最典型的现象就是机器人一上电就“打架”：各条腿之间像是在互相对抗，每条腿都像在执行一套不属于自己的控制意图。此时如果不先回头检查部署链路，而是直接怀疑 reward 或训练效果，往往会在错误方向上越走越远。

下面这几项，就是实机部署前最值得优先确认的内容。

3.1.1 坐标系 frame 解释错误

在 rl_sar 中，观测层专门保留了 `ang_vel_axis` 这个配置项，就是因为不同系统里，角速度究竟是在 body frame 还是 world frame 下表达，并不是天然一致的。框架本身支持两种处理方式：如果角速度已经是 body frame，就直接使用；如果是 world frame，就结合四元数做一次逆旋转，把它转换到机体系后再送入网络。

```
if (observation == "ang_vel")
{
    if (this->ang_vel_axis == "body")
    {
        obs_list.push_back(
            this->obs.ang_vel * this->params.Get<float>("ang_vel_scale")
        );
    }
    else if (this->ang_vel_axis == "world")
    {
        obs_list.push_back(
            QuatRotateInverse(this->obs.base_quat, this->obs.ang_vel)
            * this->params.Get<float>("ang_vel_scale")
        );
    }
}
```

这个分支看上去很小，但在真实部署中影响非常大。它决定了 policy 看到的究竟是“机体自身角速度”，还是一份被错误旋转过的量。假如机器人实际提供的已经是 body frame 角速度，但部署时仍沿用了 world frame 的处理逻辑，那么程序就会把本来正确的数据又额外做一次坐标变换。此时程序不会报错，控制循环也照样运行，SDK 连接同样看似正常，但机器人一上电就会表现出极其异常的姿态对抗。

这类问题最容易迷惑人的地方在于：它表面看起来很像“策略很差”，但本质上并不是策略没学会，而是**观测的物理意义已经错了**。

如果前面的训练是基于 robot_lab 完成的，那么需要特别注意，Isaac Lab 中的角速度观测本来就是 body frame。对应代码位于 `source/isaaclab/isaaclab/envs/mdp/observations.py`：

```
def base_ang_vel(env: ManagerBasedEnv, asset_cfg: SceneEntityCfg = SceneEntityCfg("r
    """Root angular velocity in the asset's root frame."""
    # extract the used quantities (to enable type-hinting)
    asset: RigidObject = env.scene[asset_cfg.name]
    return asset.data.root_ang_vel_b
```

这里的 `root_ang_vel_b` 中，后缀 `_b` 就表示 body/base/root frame，也就是机体根坐标系。因此在 `rl_real_d1.cpp` 中，应确认配置为：

```
this->ang_vel_axis = "body";
```

如果不是，就需要先改回来。

3.1.2 关节顺序或腿序映射错误

另一个高频问题，是关节顺序或腿序映射错误。

D1 的 SDK 状态数据与 rl_sar 内部使用的关节展开顺序是一致的，都是按照 `FR, FL, RR, RL` 展开成 12 维关节状态。如果实际使用的机器人或代码实现没有严格遵循这一顺序，就必须在 `GetState()` 和 `SetCommand()` 里显式做映射；否则，每条腿看到的就不是自己的状态，每条腿收到的也不是自己的目标。

```
// SDK关节顺序 与 rl_sar关节顺序: FR(0-2), FL(3-5), RR(6-8), RL(9-11)
static const int SDK_LEG_MAP[4] = {0, 1, 2, 3};

void RL_Real::GetState(RobotState<float> *state)
{
    this->d1_motor_state = this->d1_lowlevel.getMotorState();

    if (this->d1_motor_state != nullptr && this->d1_lowlevel.haveMotorData())
    {
        for (int leg = 0; leg < 4; ++leg)
        {
            int sdk_leg = SDK_LEG_MAP[leg];
            int base_idx = leg * 3;
```

```
            state->motor_state.q[base_idx + 0] = this->d1_motor_state->q_abad[sdk_leg
            state->motor_state.q[base_idx + 1] = this->d1_motor_state->q_hip[sdk_leg
            state->motor_state.q[base_idx + 2] = this->d1_motor_state->q_knee[sdk_leg

            state->motor_state.dq[base_idx + 0] = this->d1_motor_state->qd_abad[sdk_
            state->motor_state.dq[base_idx + 1] = this->d1_motor_state->qd_hip[sdk_le
            state->motor_state.dq[base_idx + 2] = this->d1_motor_state->qd_knee[sdk_
        }
    }
}
```

一旦映射写错，机器人同样会在上电后立刻表现出明显的关节对抗。原因并不神秘：本该发给前右腿的目标角度，被错误地送到了别的腿上；本该由某条腿反馈的位置，又被解释成了另一条腿的当前状态。对于 policy 来说，它并不知道内部线路已经“错接”了，它只会继续按照训练期的假设输出动作，最终表现出来的自然就是系统性的混乱。

3.1.3 默认姿态与训练参考姿态偏差过大

第三类经常被忽略的问题，是默认姿态与训练参考姿态差得太多。

在 rl_sar 的观测构造中，关节位置并不是直接使用绝对角度，而是先减去 `default_dof_pos`，得到相对于默认姿态的偏移量后，再送入网络。换句话说，`default_dof_pos` 不只是一个“站立初始角度”，它本质上也是 policy 理解“当前腿处于什么位置”的参考零点。

```
D1/himloco:
  observations: ["commands", "ang_vel", "gravity_vec",
                 "dof_pos", "dof_vel", "actions"]

  action_scale: [0.125, 0.25, 0.25, 0.125, 0.25, 0.25,
                 0.125, 0.25, 0.25, 0.125, 0.25, 0.25]

  lin_vel_scale: 2.0
  ang_vel_scale: 0.25
  dof_pos_scale: 1.0
  dof_vel_scale: 0.05

  default_dof_pos: [0.1, 0.8, -1.5, -0.1, 0.8, -1.5,
                    0.1, 1.0, -1.5, -0.1, 1.0, -1.5]
```

如果训练时采用的是一套默认站姿，而到了真机部署阶段却换成了另一套参考姿态，那么策略从一开始接收到的 `dof_pos` 就已经整体偏移了。此时即使通信没问题、控

制频率也正常，策略输出仍然会失真，因为它从一开始就“误解”了当前关节所处的位置。

所以，第一次实机现象出来之后，可以用一个很实用的判断标准来快速定位问题来源：

如果机器人只是**不够稳、不够顺、不够快**，但大方向、基本站姿和动作逻辑是正确的，那么通常说明部署链路大体可用，后续更多是参数微调问题；
但如果机器人表现为**上电打架、方向反了、局部腿系统性错动、甚至原地都无法按照正确逻辑响应**，那就不应该先去怀疑 reward 或训练结果，而应该立刻回头检查 frame、四元数顺序、关节映射、默认姿态以及整个观测构造过程。

3.2 编译并连接机器

当 frame、关节映射和默认姿态都确认无误之后，就可以开始编译项目并尝试连接机器人了。

首先，在 `rl_sar` 文件夹下运行以下指令，使用 CMake 编译整个项目：

```
./build.sh -m
```

接下来，通过 Wi-Fi 连接 D1 发出的 AP。这个 AP 自带 DHCP，可以给电脑分配 IP 地址。查询到本机被分配的地址后，通过 SSH 登录机器人：

```
ssh firefly@192.168.234.1    # 密码为 firefly
```

登录后，修改 SDK 的配置文件：

```
sudo vim /opt/export/config/sdk_config.yaml
```

打开后可以看到类似如下内容：

```
target_ip: "127.0.0.1"
target_port: 43988
```

此时需要将 `target_ip` 从 `127.0.0.1` 改成当前用于连接机器人那台电脑的无线网 IP。比如这里电脑的 IP 是 `192.168.234.14`，就应当改成：

```
target_ip: "192.168.234.14"
target_port: 43988
```

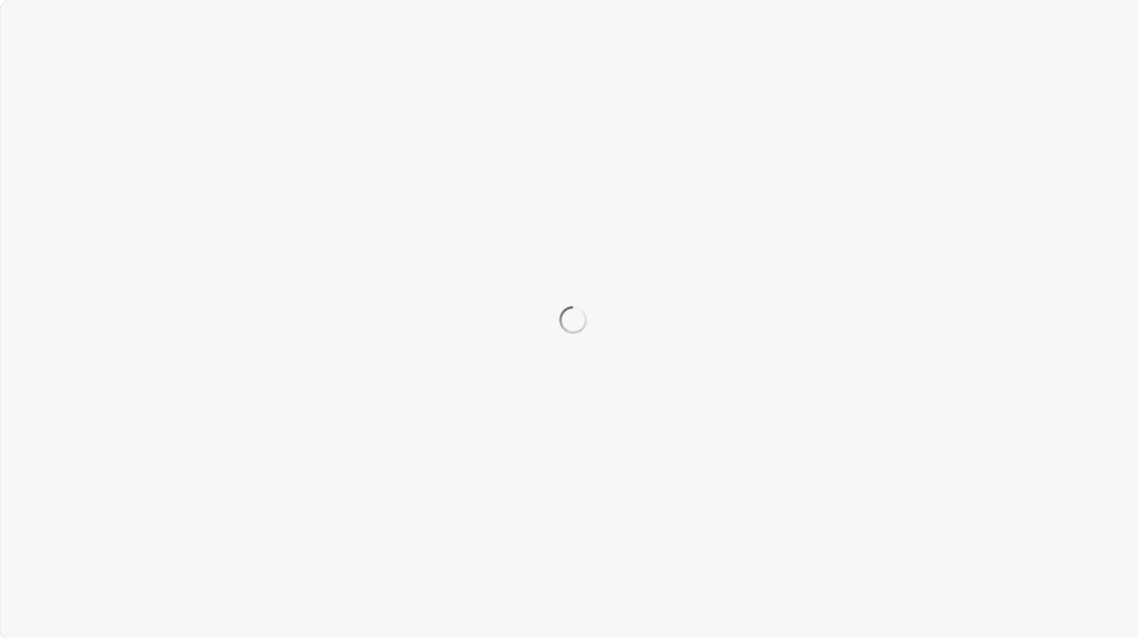
修改完成后，通过重新开关电池供电的方式重启机器人。

重启完成并重新连上之后，在电脑端运行下面的指令发起连接：

```
./cmake_build/bin/rl_real_d1 192.168.234.14 192.168.234.1
```

其中，第一个地址是电脑当前被分配到的 IP，第二个地址是 D1 本体的 IP。如果连接成功，终端会看到如下输出：

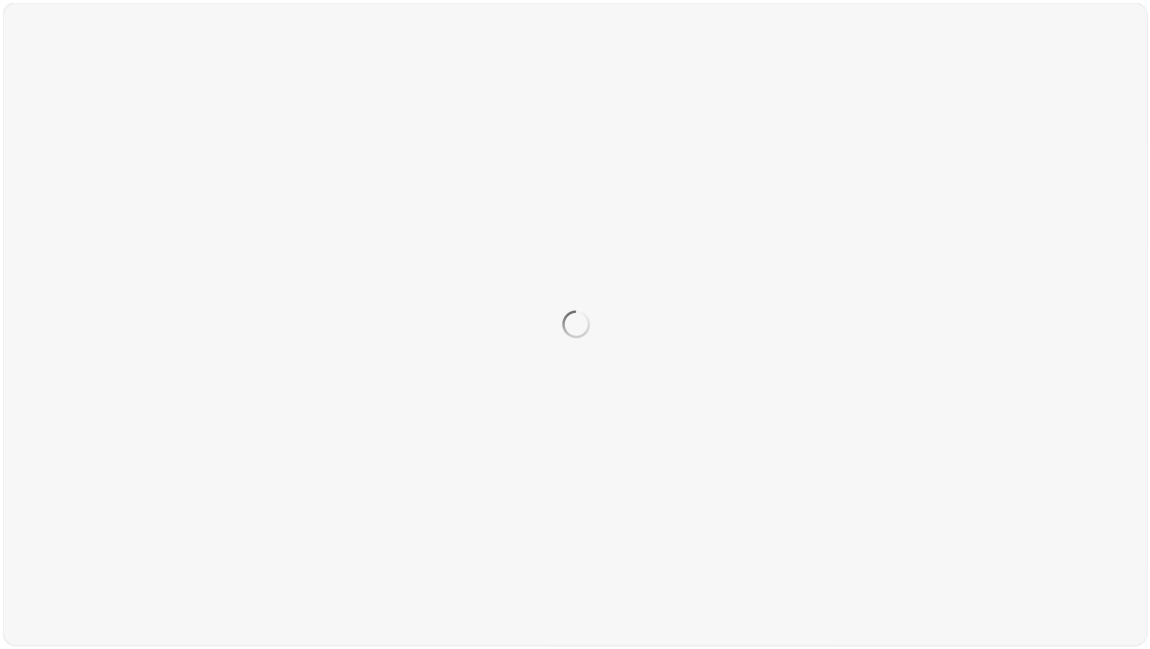
```
client bind success
connect success!
```



连接成功

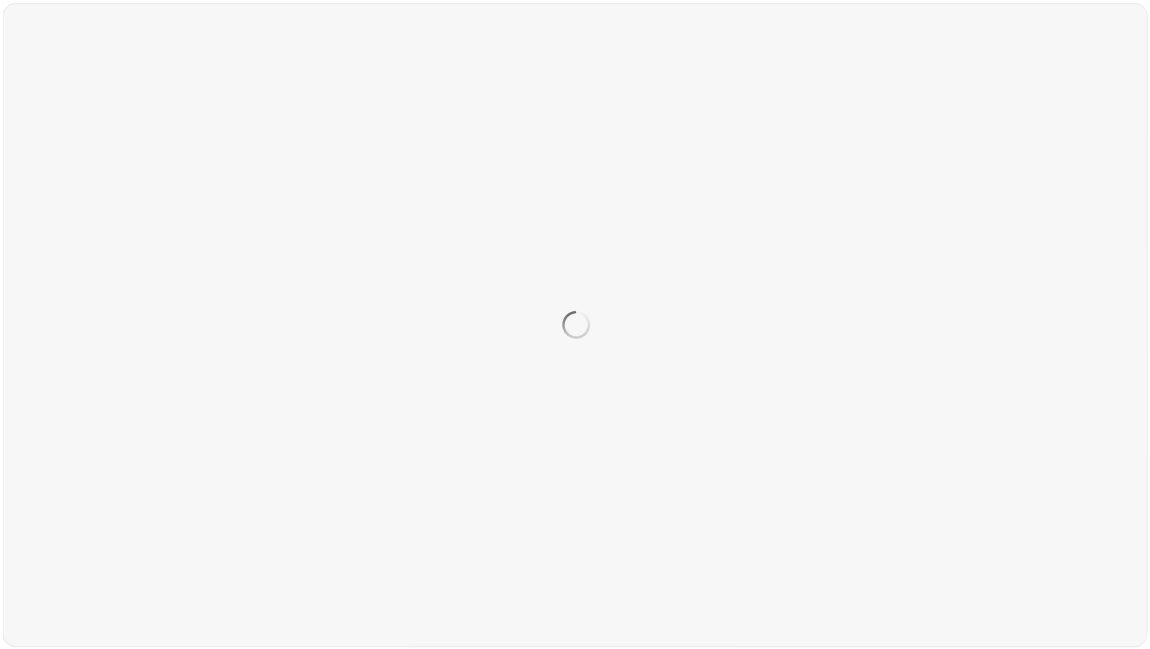
3.3 第一次用自己的策略让机器人走起来

连接成功之后，机器人会先进入阻尼模式。此时在终端中输入数字 `0`，机器人会从初始状态插值站起，并进入默认站姿：



阻尼->默认站姿

随后，在默认站姿下输入数字 `1`，机器人就会切换到强化学习控制模式。注意，在执行这一步之前，务必先与机器人保持安全距离：



强化学习控制模式

如果切换到强化学习控制后，机器人出现明显的关节对抗，也就是常说的“打架”，请第一时间输入 `P`，让机器人重新回到阻尼模式。只有在确认整体行为正常之后，才建议继续输入 `W`，给出前进速度指令，先尝试让机器人以较慢速度前进 1～3 次，观察其基本运动表现。

下面是控制键位表：

键盘控制	功能描述
0	起立：从初始姿态插值移动到 `default_dof_pos`

键盘控制	功能描述
P	趴下：电机直接进入阻尼模式
1	RL 控制：后续动作由强化学习策略接管
W	增加 X 方向速度，每按一次 +0.1 m/s
S	减少 X 方向速度，每按一次 -0.1 m/s
A	增加 Y 方向速度，每按一次 +0.1 m/s
D	减少 Y 方向速度，每按一次 -0.1 m/s
Q	增加 Yaw 方向角速度，每按一次 +0.1 rad/s
E	减少 Yaw 方向角速度，每按一次 -0.1 rad/s

到这一步，Sim2Real 这件事才算真正“开始发生”。因为从这里开始，看到的每一个现象——是站得稳，还是腿发软；是动作太猛，还是响应迟钝；是能前进但不平顺，还是一切正常——都将直接决定后面的调整方向。

4. 表现不如意？先从部署侧微调开始

第一次把策略接到 D1 上之后，只要已经确认通信正常、状态在更新、控制链条大体闭合，下一步通常不应该立刻回去重训。更有效的做法，往往是先在部署侧做几项**小范围、强可解释、并且能快速回退**的修正。

因为第一次实机现象中，通常混杂着两类因素：一类来自策略本身，说明训练结果仍有提升空间；另一类则来自部署侧参数还没有把策略放进一个适合真实机器人发挥的工作区间。如果这两类问题一开始就混在一起处理，就很容易陷入“越调越乱”的局面。

从 rl_sar 的动作输出链条来看，这套逻辑其实非常清楚：网络先输出归一化动作，随后乘上 `action_scale`，再叠加 `default_dof_pos` 得到目标关节位置，最后配合 `rl_kp`、`rl_kd` 形成 PD 形式的控制输出。也就是说，部署初期最值得优先关注的几个“旋钮”，其实就集中在 `action_scale`、`rl_kp`、`rl_kd` 和命令范围这几项上。

```
void RL::ComputeOutput(const std::vector<float> &actions,
                       std::vector<float> &output_dof_pos,
                       std::vector<float> &output_dof_vel,
                       std::vector<float> &output_dof_tau)
{
    // 1. 动作缩放
```

```
    std::vector<float> actions_scaled =
        actions * this->params.Get<std::vector<float>>("action_scale");

    // 2. 叠加默认姿态，得到目标位置
    output_dof_pos =
        actions_scaled + this->params.Get<std::vector<float>>("default_dof_pos");

    // 3. 位置-速度形式的PD控制输出
    output_dof_tau =
        this->params.Get<std::vector<float>>("rl_kp")
        * (actions_scaled
           + this->params.Get<std::vector<float>>("default_dof_pos")
           - this->obs.dof_pos)
        - this->params.Get<std::vector<float>>("rl_kd") * this->obs.dof_vel;

    // 4. 力矩限幅
    output_dof_tau = clamp(
        output_dof_tau,
        -this->params.Get<std::vector<float>>("torque_limits"),
        this->params.Get<std::vector<float>>("torque_limits")
    );
}
```

从这段代码不难看出，如果第一次实机调试时机器人动作明显过猛、落脚很砸、停下来之后还在持续发力，那么最先该怀疑的往往不是“策略不会走”，而是输出链条在真机上过于激进了。比如：

- `action_scale` 太大，会把每一维动作在真实关节上的幅度放得过多；
- `rl_kp` 太高而 `rl_kd` 不足，会让动作表现得很硬、很紧，误差一出现就会用更大的力去追；
- 力矩限制过宽，则会让这些问题在真机上表现得更直接。

与其一开始就在训练侧大动干戈，不如先把输出区间收窄，让机器人进入一个更温和、更可观察的运行状态。只有在这个基础上，后续的现象才更容易被正确解释。

下面这段配置，就是部署侧微调时最常接触到的一组关键参数：

```
D1:
  dt: 0.005
  decimation: 4

  fixed_kp: [80.0, 80.0, 80.0, 80.0, 80.0, 80.0,
             80.0, 80.0, 80.0, 80.0, 80.0, 80.0]
  fixed_kd: [3.0, 3.0, 3.0, 3.0, 3.0, 3.0,
             3.0, 3.0, 3.0, 3.0, 3.0, 3.0]

  torque_limits: [23.5, 23.5, 23.5, 23.5, 23.5, 23.5,
                  23.5, 23.5, 23.5, 23.5, 23.5, 23.5]
```

```
  default_dof_pos: [0.00, 0.80, -1.50, 0.00, 0.80, -1.50,
                    0.00, 0.80, -1.50, 0.00, 0.80, -1.50]

D1/himloco:
  rl_kp: [40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40]
  rl_kd: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

  action_scale: [0.125, 0.25, 0.25, 0.125, 0.25, 0.25,
                 0.125, 0.25, 0.25, 0.125, 0.25, 0.25]

  commands_scale: [2.0, 2.0, 0.25]

  default_dof_pos: [0.1, 0.8, -1.5, -0.1, 0.8, -1.5,
                    0.1, 1.0, -1.5, -0.1, 1.0, -1.5]
```

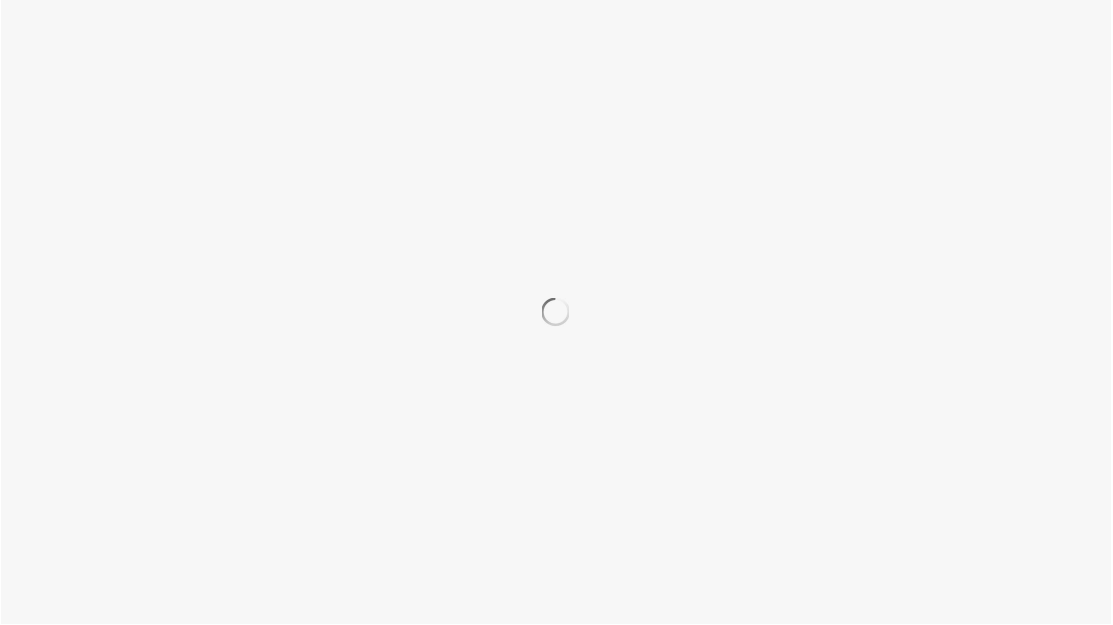
下面结合几类典型现象，看看这些参数是如何用于部署侧微调的。

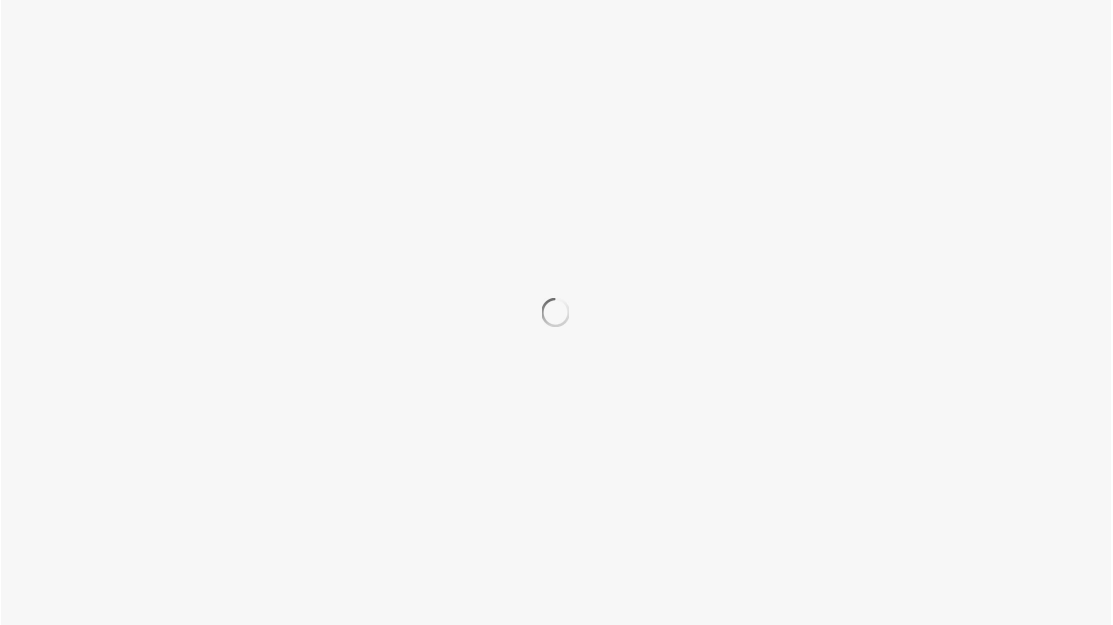
4.1 运动指令不敏感

所谓“运动指令不敏感”，指的是机器人实际表现出来的运动速度明显低于期望速度。常见原因之一，是 `clip_actions` 的上下限过早限制了策略动作的发挥，导致策略虽然“想走得更快”，但输出在进入控制链之前就已经被截断了。

```
  clip_actions_lower: [-3.0, -3.0, -3.0,
                       -3.0, -3.0, -3.0,
                       -3.0, -3.0, -3.0,
                       -3.0, -3.0, -3.0]
  clip_actions_upper: [3.0, 3.0, 3.0,
                       3.0, 3.0, 3.0,
                       3.0, 3.0, 3.0,
                       3.0, 3.0, 3.0]
```

例如，把上下限从 `-1.6 ~ 1.6` 提高到 `-3.0 ~ 3.0` 之后，在同样是 `0.5 m/s` 的速度指令下，机器人的运动意愿通常会更明显。这说明问题并不一定出在策略不会跟踪速度，而可能只是输出在部署侧被限制得过早了。





4.2 “腿软”

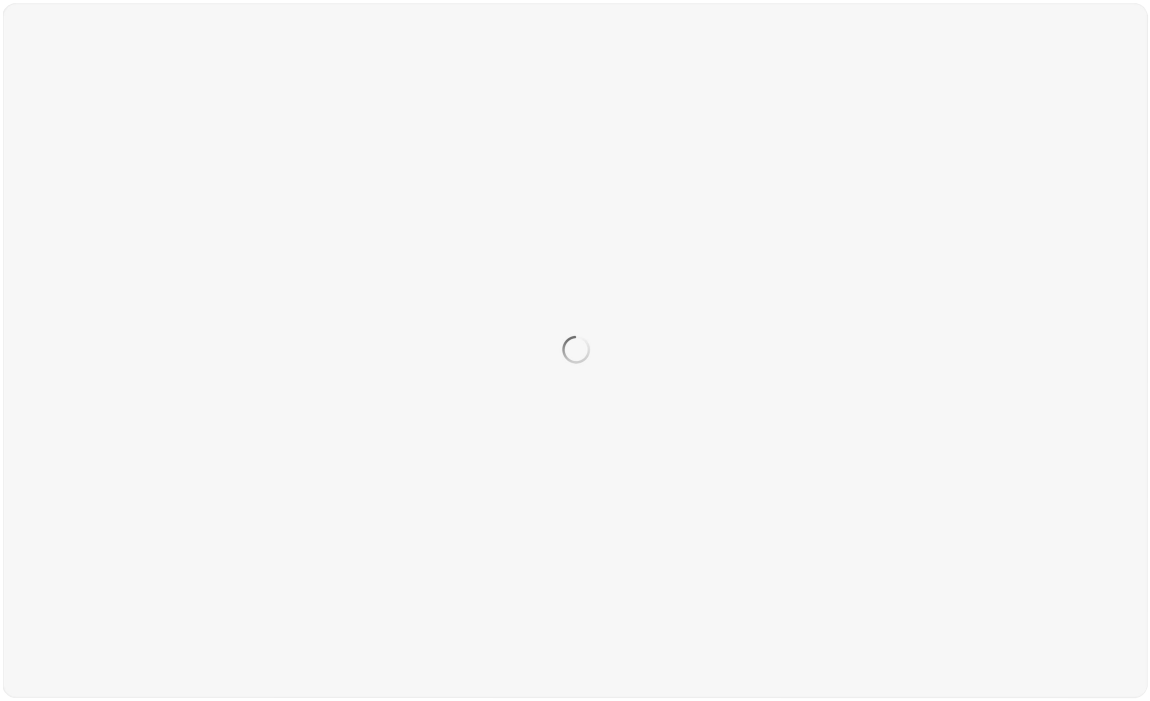
“腿软”是第一次实机中非常常见的一类现象。直观表现为机器人有时无法稳定支撑自己，尤其是在后退或姿态变化较大时，更容易出现下沉、发虚，甚至有摔倒倾向。

这类现象通常意味着关节刚度不足，也可能意味着阻尼偏小，导致动作在受到扰动后容易回弹、不够稳。因此，优先调整的通常是 `rl_kp` 和 `rl_kd` 两个参数。

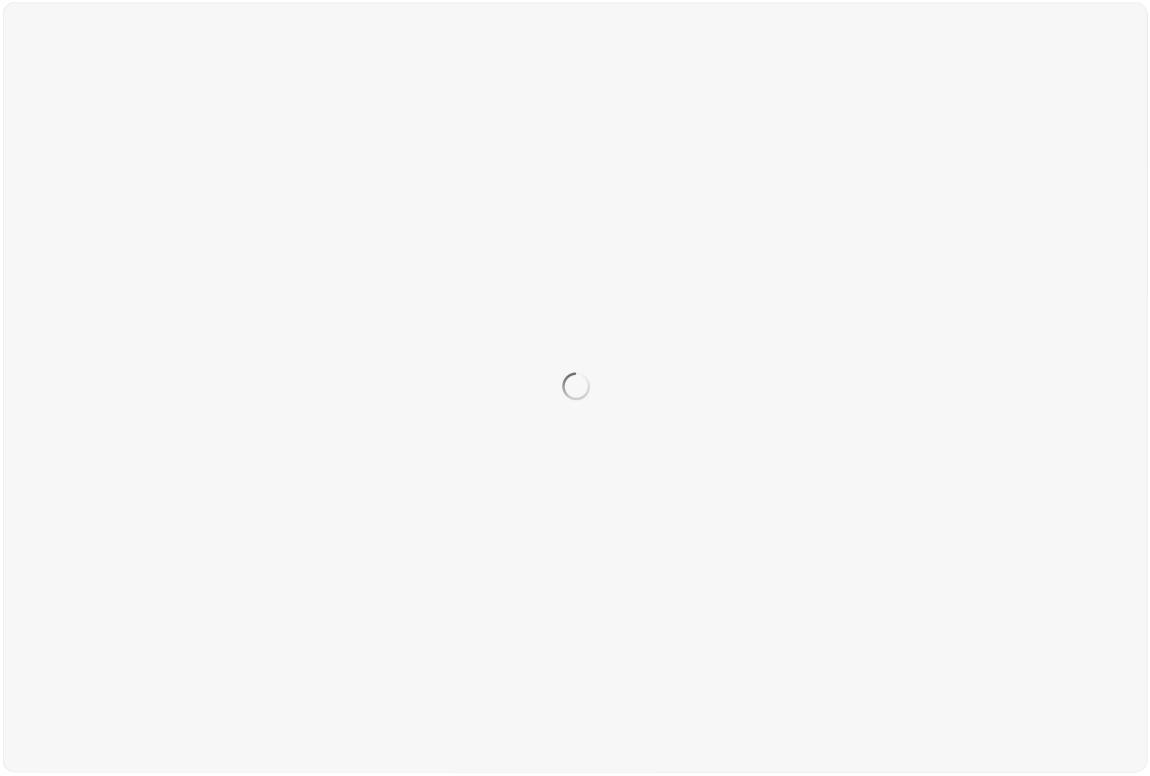
```
rl_kp: [18.0, 18.0, 18.0,
        18.0, 18.0, 18.0,
        18.0, 18.0, 18.0,
        18.0, 18.0, 18.0]
rl_kd: [0.5, 0.5, 0.5,
        0.5, 0.5, 0.5,
```

```
            0.5, 0.5, 0.5,
            0.5, 0.5, 0.5]
```

从控制意义上说，`rl_kp` 决定了“偏了之后要多努力地拉回来”，`rl_kd` 则决定了“在拉回来的过程中要不要多一点阻尼抑制”。所以遇到“腿软”时，不是简单理解成“腿没力气”，而更应该从控制器的刚度和阻尼两个维度去看。



调整前



调整后

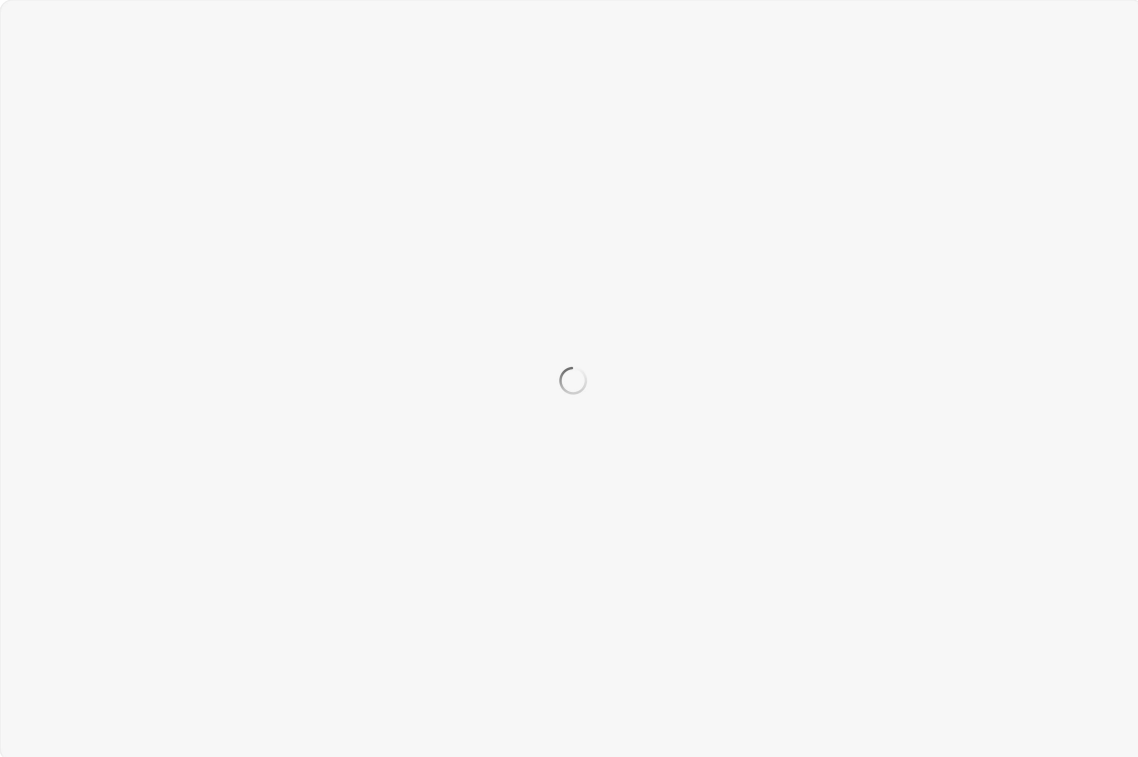
4.3 抬腿高度不及预期

还有一种很典型的现象，是机器人能走，但抬腿高度明显不够。这样一来，不仅越障能力会受影响，很多时候连基本的运动速度也会受限，因为步态幅度没有真正释放出来。

这时通常可以优先检查 `action_scale`。如果这一项设置得过小，那么网络输出虽然有动作意图，但实际映射到关节上的幅度依然偏保守，最终表现出来的就是“动作有了，但幅度没出来”。

```
action_scale: [0.125, 0.25, 0.25,
               0.125, 0.25, 0.25,
               0.125, 0.25, 0.25,
               0.125, 0.25, 0.25]
```

适当调大 `action_scale` 之后，能够释放更多网络输出，让关节摆动幅度变大，进而改善抬腿高度、步态张力与整体通过性。当然，这项参数也不宜一次调得过猛，否则又可能反过来带来动作过激的问题。

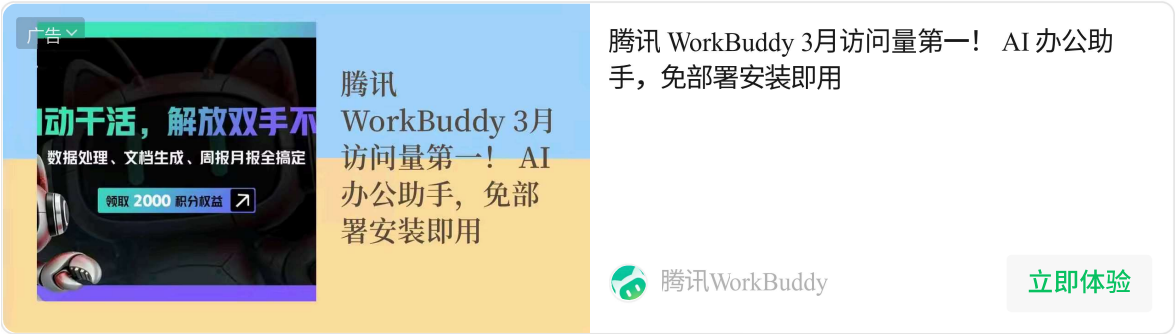


抬腿高度较低

调整后的抬腿高度

上篇小结

到这里，上篇的重点内容就基本完整了。对于智元 D1 的 Sim2Real 落地来说，真正关键的第一步，并不是“把模型跑起来”，而是先把控制链路理顺：明确 Lowlevel 的接入方式，理解 SDK 的上下行数据流，建立策略环与控制环的双频结构，完成部署前的关键检查，并在第一次实机之后优先通过部署侧参数进行收敛式微调。



很多时候，机器人第一次表现不理想，并不意味着训练失败，而只是说明策略还没有被放到一个适合真实硬件发挥的工作区间里。下篇我们会继续进入训练侧分析，包括如何结合 TensorBoard 判断策略问题、如何重新定位优化方向，以及如何进一步尝试在 Jetson Orin 上完成嵌入式部署。

点点关注点点赞，发现更多自动驾驶/具身智能/GitHub 有趣内容！ 🚀

敢敢AUTOHUB
致力于分享一线具身智能/自动驾驶相关知识，希望能够一起不断学习成长
420篇原创内容
公众号

往期内容回顾 👀

📑 🔥 十分钟实用教程 | Ralph Loop：Claude Code 自主迭代机制的深度解析
📑 🔥 十分钟读论文 | Navigation World Models: 构建机器人视觉导航的“想象力引擎”
📑 🔥 行业杂谈 | AI Infra 深度解析：从算力底座到智能应用的技术桥梁

喜欢作者

智元D1开发实战 · 目录 ≡

< 上一篇
智元D1开发教程 | 强化学习怎么验证？Sim2Sim框架精讲！

下一篇 >
智元D1开发教程 | Sim2Real全指南（下）——从训练诊断到 Jetson Orin 实机部署

阅读 243



全新四代ETC，磁吸插卡，微信扣费，缺电可刷卡过闸，不用贴玻璃

ETC助手

了解更多